

Введение в экосистему Solana

Занятие 1

Содержание

- **Лекция (Теория):**
 - Знакомство
 - Обзор Solana: история, архитектура и экосистема
 - Сравнение Solana с другими блокчейнами
 - Механизм консенсуса Solana: Proof of History
 - Как PoH влияет на масштабируемость, безопасность и производительность
 - EVM vs BPF. Почему стоимость транзакций дешевле чем в других блокчейнах?
 - Почему Rust важен для разработки на Solana и ключевые особенности Rust, полезные для разработки блокчейн-приложений

Обо мне

Дмитрий Елисов

Окончил МФТИ в 2014

В крипте с 2017

Управлял фермой на 600 устройств

Основатель компании Oqtacore

70 человек, занимаемся заказной разработкой для маленьких и больших компаний: Bitcoin, EVM, Solana, Tron, Ton, Hyperledger

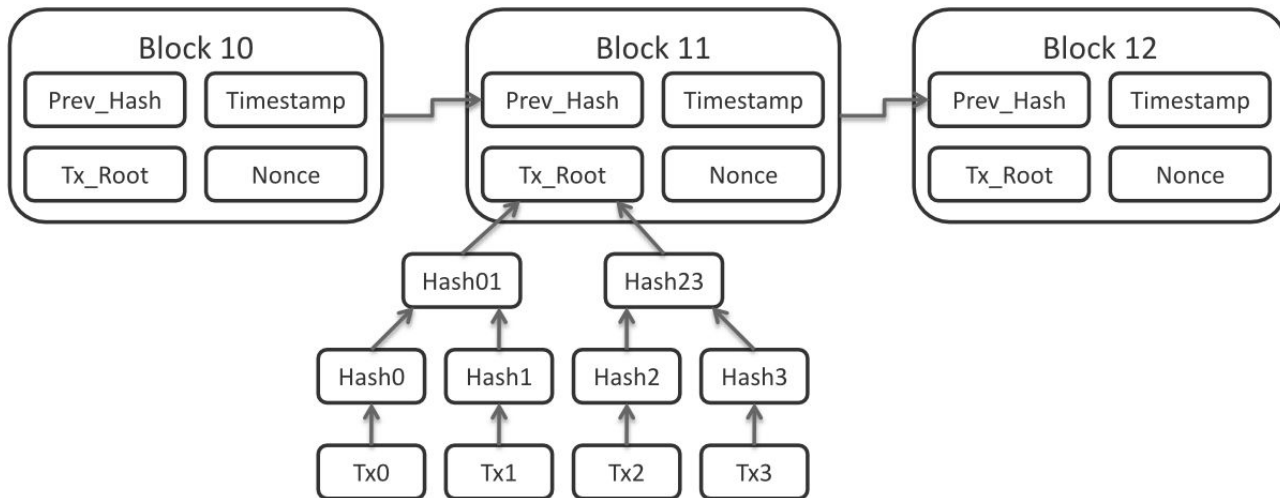


О вас

- Кто программировал на Rust?
- Кто программировал на солане?
-

Зачем Solana вообще появилась

- Проблема: низкая пропускная способность и высокая задержка L1-блокчейнов
- Цель Solana: высокая пропускная способность + низкая latency
- Никаких L2, роллапов, сайдчейнов, воркчейнов, парачейнов и т.п.
- Подход: оптимизация времени, сети и параллелизма, а не только консенсуса



Зачем Solana вообще появилась

- Решение

Блоки должны генерироваться с максимально возможной скоростью. Давайте сделаем так, что блоки будут появляться максимально часто несмотря ни на что. Основной фактор, ограничивающий пропускную способность сети, — это не мощность CPU, а эффективность достижения консенсуса. Валидатор, который создаст блок, может быть вычислен, и известен заранее. Задача валидатора — включить в свой блок столько транзакций, сколько он успеет обработать в пределах отведенного времени.

Краткая история

- 2017 — идея Proof of History (Анатолий Яковенко)
- 2018–2019 — первые прототипы, Solana Labs
- 2020 — mainnet beta
- 2021–2022 — рост DeFi/NFT, стресс-тесты сети
- 2023–2024 — фокус на стабильность, клиентское разнообразие, Firedancer

Ключевая идея Solana

- Proof of History (PoH) = криптографическая временная шкала
- Привязываемся не к реальному времени, а создаем свои собственные “тики”
- Следующий “тик” получается из предыдущего с помощью SHA256
- Слот - 64 тика
- На время слот лидер валидирует транзакции и транслирует их дальше в сеть
- Консенсус и исполнение не блокируют друг друга
- Лидер стримит транзакции, а не отправляет их в сеть одним блоком

Proof of History (PoH)

- Последовательное хеширование (SHA-256)
- Каждое событие привязано к позиции во времени
- Узлы могут проверять порядок событий без общения между собой
- PoH — это независимый генератор времени. Данные не создают время, данные лишь занимают место во времени.
- В других блокчейнах хеши блоков зависят от данных в блоке. В солане не так.

Что это даёт:

- Быстрая валидация
- Меньше сетевого шума
- Предсказуемый порядок транзакций

PoH не гарантирует:

- корректность данных
- валидность транзакций
- честность лидера

Задача византийских генералов

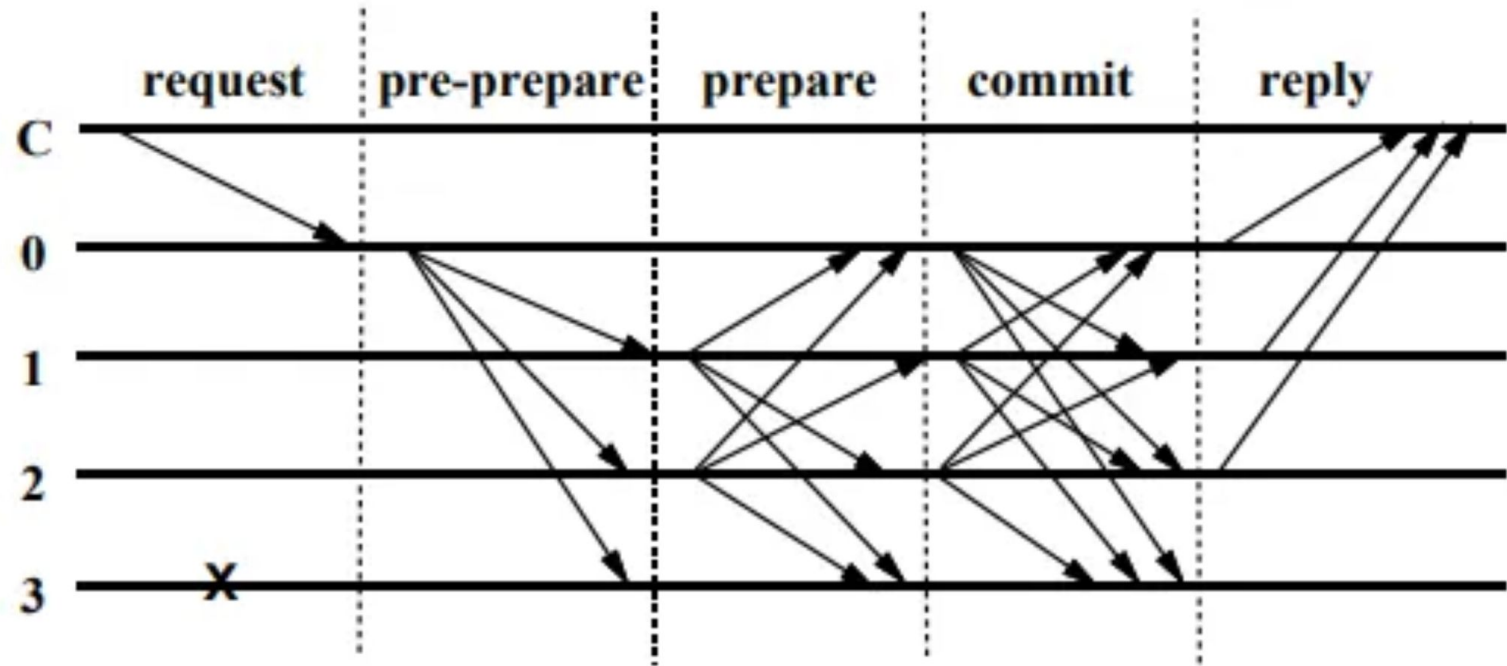
- Имеется N генералов, окруживших город
- Часть из них - предатели. Могут врать, молчать.
- Гонцы от честных генералов могут не доходить/подменяться

Победа возможна, только если удастся достичь консенсуса

Добиваемся двух свойств:

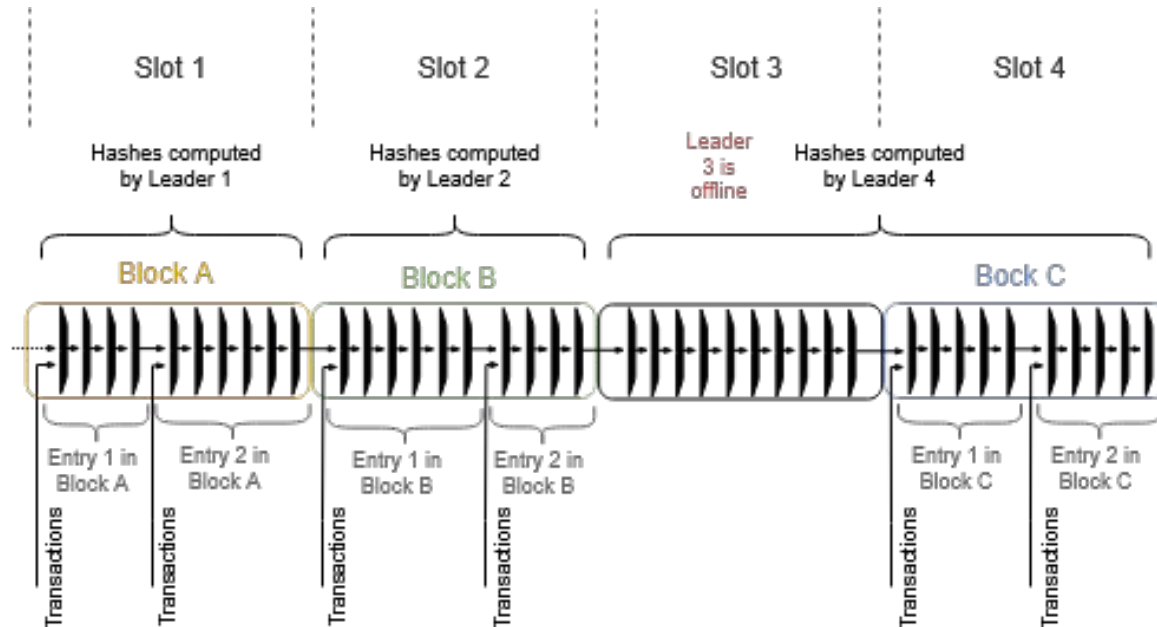
- Agreement (согласие) - сходимость к единому решению
- Validity (корректность) - консенсусу принимает решение **честных** генералов

Консенсус: pBFT



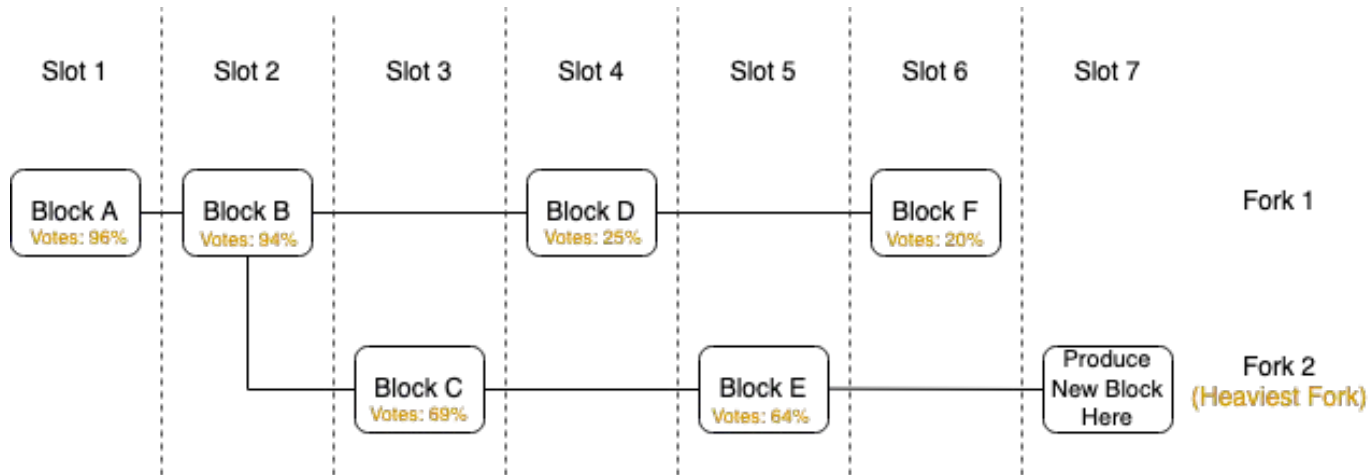
Консенсус в солане: tower BFT

- Основан на PBFT
- Использует PoH как источник времени. Слот длится примерно 0,4 сек



Консенсус в солане: tower BFT

Каждому слоту назначается определенный валидатор. Назначенный валидатор, известный как лидер, отвечает за предложение нового блока, содержащего транзакции, в течение своего назначенного слота. После того, как блок предложен, другие валидаторы в сети голосуют за его действительность, что в конечном итоге приводит к подтверждению блока и его включению в блокчейн.



Консенсус в солане: tower BFT

В Solana лидер стримит транзакции, а блок — это то, что сеть из этой истории восстанавливает.

Причины:

— **latency**: не ждём сборки блока, валидаторы начинают проверку **сразу**

— **throughput**: поток легче масштабировать, меньше пиковых нагрузок

— **отказоустойчивость**: потерял пакет → восстановил из соседей, не нужен «один большой объект»

[PoH tick]
[tx1, tx2, tx3]
[PoH tick]
[tx4]
[PoH tick]

Голоса в солане: tower BFT

Голос - это такая же транзакция. Лидер стримит голоса за прошлые слоты, потому что это повышает надежность его ветки и позволяет избежать lockout

Валидатор голосует не за транзакцию и не за блок. Валидатор голосует за всю историю до слота N

Как PoH влияет на масштабируемость, безопасность и производительность

Масштабируемость

- PoH убирает необходимость **сетевого согласования порядка транзакций**
- Порядок событий становится **вычислимым**, а не результатом обмена сообщениями
- Лидер может работать через несколько RPC одновременно
- Отказ от глобального mempool
- Валидаторы могут “опаздывать” и это не страшно
- Параллельное исполнение транзакций (Sealevel)

clients → ingress nodes → leader core → PoH → Turbine

Масштабирование достигается **за счёт CPU и сети**, а не за счёт дополнительных уровней (L2)

Как PoH влияет на масштабируемость, безопасность и производительность

Производительность

- Транзакции стримятся и проверяются **по ходу**, без ожидания сборки блока
- Валидаторы начинают верификацию **немедленно**, снижая latency

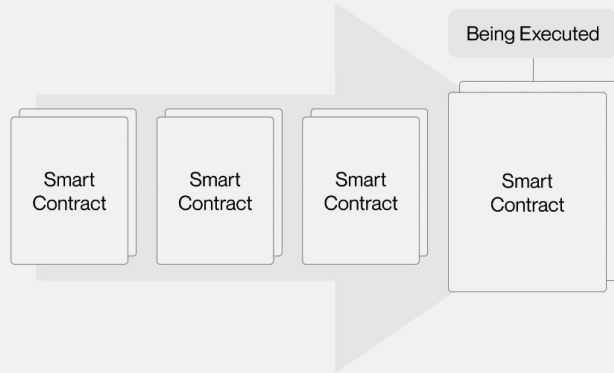
Как PoH влияет на масштабируемость, безопасность и производительность

Безопасность

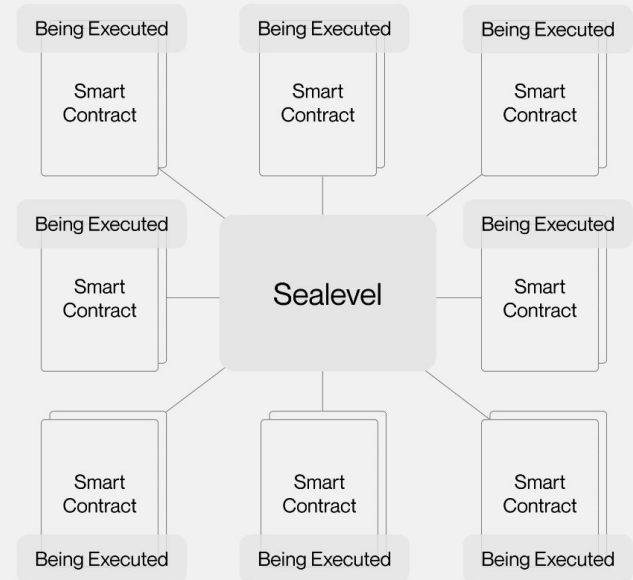
- PoH **не заменяет консенсус**, а усиливает его
- Делает попытки: reordering'инга **проверяемыми без общения**
- Снижает attack surface: меньше сложных сетевых состояний, меньше race-condition'ов
- Безопасность данных и финализация обеспечиваются: проверкой транзакций, Tower BFT

Sealevel

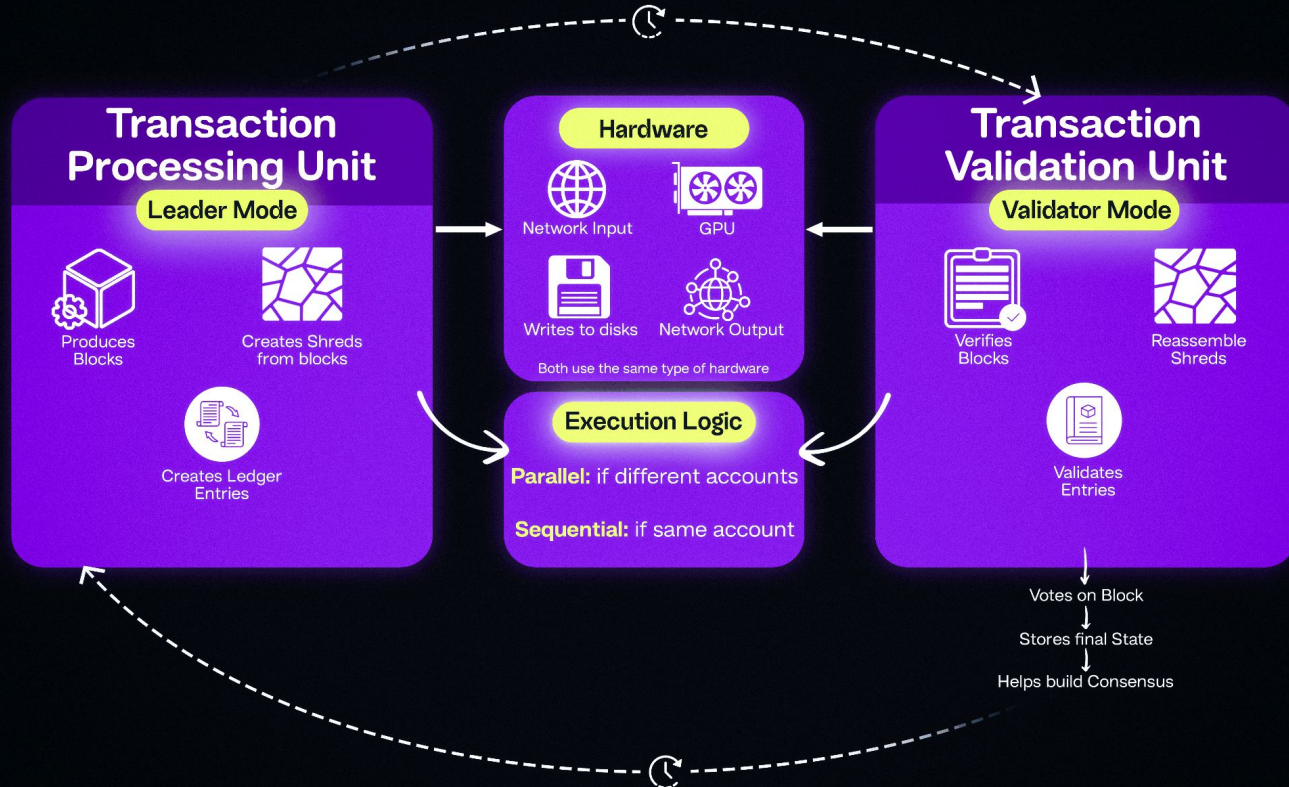
Single-threaded runtime



Multi-threaded runtime



Solana Validator



Solana validators switch between two roles: TPU (Transaction Processing Unit) when they're the leader and create blocks, and TVU (Transaction Validation Unit) when they're not the leader and validate blocks. Transactions are processed in parallel if they affect different accounts, and sequentially if they affect the same one.

EVM vs BPF. Почему стоимость транзакций дешевле чем в других блокчейнах?

EVM

- Последовательное исполнение транзакций
- Глобальное состояние
- Конфликты обнаруживаются в runtime
- Каждая транзакция блокирует весь runtime
- Даже независимые операции исполняются по очереди

BPF (Solana)

- Параллельное исполнение (Sealevel)
- Явно заданные read/write аккаунты
- Конфликты известны до исполнения
- Независимые транзакции исполняются одновременно
- CPU используется эффективно, а не простаивает

Почему Rust важен для разработки на Solana

Соответствие архитектуре Solana

- Solana исполняет программы в **BPF runtime**, близком к нативному CPU
- Rust компилируется в **детерминированный, оптимизированный байткод**
- Минимальный runtime-overhead по сравнению с VM-языками
- Предсказуемая производительность при массовом исполнении

Изучаем Solana Explorer

Задача на дом

- Установить **Rust** на локальные машины.
- Установить Solana CLI на локальные машины.